Energy-Aware Priority Scheduling in Smart Sensor Networks Using Policy-Guided Deep Reinforcement Learning

Anand Emmanuel Raju B.[1], Radhika N.[1], and Radhika G.[1]

Department of Computer Science and Engineering,
Amrita School of Computing, Coimbatore,
Amrita Vishwa Vidyapeetham, India
cb.sc.p2cse25003@cb.students.amrita.edu, n_radhika@cb.amrita.edu,
g_radhika@cb.amrita.edu

Abstract. Smart Grid sensor networks have a basic problem: ensuring reliable delivery of protection critical information (fault notifications and breaker commands) to the operators while maintaining continuous telemetry channels (voltage, current, temperature). Scheduling with Strict Priority policy—which always transmits alarms first—guarantees better latency performance for the critical data but prevents telemetry from working during disturbances, which is crucial at those times. In this paper, we explore how reinforcement learning can solve competing scheduling priorities problem better than heuristic-based approaches. A Double-DQN policy was trained in the environment created by ns-3 simulator and PyTorch and evaluated on 20 matched runs. We show that there is an obvious trade-off: critical packets transmit 5.2 faster with Strict Priority policy (0.0274 seconds vs. 0.1431 seconds), but telemetry lags about twice longer (0.6171 seconds vs. 0.4287 seconds). The proposed learned policy provides a latency improvement of 30.5% at a cost of 2.56% degradation of packet delivery ratio.

Keywords: Smart Energy Systems · Wireless Sensor Networks · Reinforcement Learning · ns-3 · QoS · Priority Scheduling

1 Introduction

At the distribution side, the power network highly relies on wireless sensor networks to carry out the following tasks: detection of faulty equipment, monitoring the condition of the network in real time, and remote operation of breakers. In reality, each gateway sensor is required to concurrently process two different types of data traffic. The first one is critical in nature, comprising protection alarms that require millisecond delivery. The second one is non-critical, and comprises telemetry (voltage, current, temperature).

The systems employed today employ Strict Priority scheduling in handling this issue. The simple principle behind the scheduling is that critical packets should always come first while routine telemetry should be sent only if there are

no packets in the critical queue. It works quite well in the case of critical alarms as the operators do get their alarms. However, in the case of a power grid failure, where the protection alarms occur heavily and continuously for a few seconds, the routine telemetry queue gets totally starved out.

Is there a way machine learning could do better? The answer seems quite obvious: rather than following a hardcoded rule, train the gateway to come up with an adaptive policy that would balance the two traffic goals. It is possible that the learning system will realize that the recently received critical message should not be forwarded immediately since its deadline is still 500 milliseconds ahead of time, that is the moment when it could flush out old telemetry packets before they become too outdated. However, this is not an easy task to accomplish.

Our study explicitly attempts to solve this problem. We used a Double DQN algorithm for scheduling with explicit reward shaping to prevent collapse, trained it in ns-3 simulation combined with PyTorch, and conducted 20 matched experiments against Strict Priority and Random algorithms. The important observation that we made is straightforward: our algorithm does not surpass Strict Priority in terms of alert latency. Critical packets are delivered in 0.1431 seconds via our algorithm, compared to 0.0274 seconds via Strict Priority. However, there is a trade-off here. Our scheduler delivers telemetry data with 30.5% lower latency (0.4287 seconds compared to 0.6171 seconds).

The core contributions include:

- Fully reproducible simulation framework that integrates ns-3.40 and PyTorch for a direct comparison of policies in a controllable environment. The framework is open source and extendable.
- Guidance based on a soft reward system that avoids policy breakdowns, while preserving flexibility towards the attainment of dual objectives. Intensity of guidance depends on the age of the packets, being less intensive if deadlines are far and more intensive near expiration.
- Empirical results proving that the learned strategy generalizes well for three network sizes (50, 100, 200 nodes), although success varies according to the situation.
- Tradeoffs Quantified: 2.56% reduction in mission-critical delivery (98.34% to 95.78%) in exchange for 30.5% decrease in telemetry delay. The ratio is skewed towards operators who require awareness during grid disturbance conditions.

2 Related Work

Application of reinforcement learning to wireless communications has been made within the last decade. This is due to the simple reason that instead of manual tuning of communication protocols, one would be able to adjust to changing circumstances using learning-based approaches. Surveys [3,1] on these topics consider spectrum allocation, routing, MAC protocol design, and packet scheduling. Nonetheless, both surveys highlight the same problem - the lack of guarantee of safety in the process of RL and large amounts of data needed for training.

For the application of sensor networks, there are a number of implementation and analysis approaches that touch on the issue. According to Leong [2], neural policies are able to adapt to the distribution of packets arriving in the network and fading behavior which is difficult for any fixed heuristics to capture. The application of distributed RL in order to achieve coverage and lifetime has been considered by Sharma and Chauhan [4].

However, there is one area where a gap exists within the literature, namely a direct comparison between the two types of scheduler in question. While most tests of learning compare learning against random allocation and no control at all, we will be filling that gap through comparing the two scheduling strategies, with an eye towards understanding the tradeoffs between the two.

3 System Model and Problem Formulation

3.1 Network and Traffic Model

A sensing cluster that represents an edge device of a smart grid distribution substation is developed. The cluster consists of 100 wireless sensors randomly distributed over an area of 150 m × 150 m and communicating with a central gateway. The physical and MAC layers employ ns-3.40 implementation of IEEE 802.15.4 in the 2.4 GHz band.

Sensor nodes generate two distinct traffic classes: High-Priority (HP) packets representing critical protection alerts and fault notifications, and Normal-Priority (NP) packets representing telemetry and state-estimation feeds. HP generation follows an unannounced bursty Poisson process; NP traffic is periodic. Each node maintains separate FIFO buffers (HP capacity: 50 packets, NP capacity: 100). Packets exceeding deadlines ($T_H^{\max} = 1.0\,\mathrm{s}$, $T_N^{\max} = 3.0\,\mathrm{s}$) are dropped. The gateway scheduler operates at 100 ms decision intervals over 5000-step episodes (500 s real time).

3.2 MDP Definition

Scheduling is formulated as a discounted infinite-horizon MDP.

$$\mathbf{s}_t = \left[q_t^N,\ q_t^H,\ e_t,\ a_t^N,\ a_t^H,\ d_t\right], \tag{1}$$

where $q_t^N, q_t^H \in [0,1]$ denote normalized buffer occupancy; e_t denotes normalized residual energy; $a_t^N, a_t^H \in [0,1]$ denote normalized head-of-line packet ages relative to expiration; and d_t denotes normalized channel contention.

The action space comprises five discrete choices:

$$\mathcal{A} = \{\text{send HP (0)},\ \text{send normal (1)},\ \text{drop normal (2)},\ \text{sleep (3)},\ \text{drop HP (4)}\}. \tag{2}$$

Actions 0, 1 attempt transmission; Action 3 enters sleep; Actions 2, 4 enable buffer purging of aged packets.

3.3 Reward Formulation with Soft Policy Guidance

The reward mechanism faces the following challenge: the reward mechanism based on simple throughput produces policy collapse (failure to take into account important traffic), whereas punishment makes it impossible to avoid replicating the Strict Priority approach. Solution:

$$r_t = \tanh\left(\frac{r_t^{\text{base}} + r_t^{\text{guide}}}{100}\right), \tag{3}$$

Where the base reward takes into account multiple criteria (successful deliveries with weights depending on classes $w_1 = 25.0$ for HP, $w_2 = 5.0$ for NP; penalization for packet drops and timeouts; penalization for latency; energy consumption). The guidance reward provides reinforcement for HP delivery and negative feedback for late HP packets only if urgent ($a_t^H > \tau_u = 0.55$), leaving flexibility for trading off between the two. The base part takes into account the main network metrics:

$$r_t^{\text{base}} = w_1 N_{\text{sent}}^H + w_2 N_{\text{sent}}^N + w_3 N_{\text{drop}}^H + w_4 N_{\text{drop}}^N + w_5 N_{\text{to}}^H + w_6 N_{\text{to}}^N + w_7 (a_t^H)^2 + w_8 (a_t^N)^2 - w_9 \Delta e_t, \tag{4}$$

where N_{sent}^H and N_{sent}^N represent confirmed deliveries ($w_1 = 25.0, w_2 = 5.0$); N_{drop} and N_{to} penalize buffer overflow drops and timeout expirations ($w_3 = -40.0, w_4 = -8.0, w_5 = -80.0, w_6 = -10.0$); quadratic terms penalize waiting queue latencies ($w_7 = -40.0, w_8 = -2.0$); and Δe_t penalizes energy depletion ($w_9 = 250.0$). If the task is chosen to be sleep (Action 3), the scheduler gets a conservation reward $r_{\text{sleep}} = +0.5$.

In order to guarantee the agent services important traffic without losing its operational flexibility, we propose the soft guidance policy term:

$$r_t^{\text{guide}} = \begin{cases} +\alpha, & a_t = 0 \text{ and } q_t^H > 0 \\ -\beta, & a_t \neq 0,\ q_t^H > 0,\ a_t^H > \tau_u \\ 0, & \text{otherwise} \end{cases} \tag{5}$$

with bonus $\alpha = 6.0$, penalty $\beta = 8.0$, and urgency age threshold $\tau_u = 0.55$. Under such a formulation, the agent gets positive rewards for delivering the HP packets. Importantly, the penalty β for not doing the task is not levied when the waiting HP packet is young ($a_t^H \leq \tau_u$).This design grants the scheduler autonomy to temporarily withhold a newly arrived HP packet to flush an aging normal queue, intervening with negative feedback only when an HP packet approaches critical delay boundaries.

4 Methodology

4.1 Learning Architecture

We implement Double Deep Q-Network (Double-DQN) to mitigate maximization bias in standard Q-learning. The target computation splits action selection,

through online network θ_t, from evaluation, using target network θ_t^-.

$$Y_t^{\text{DoubleQ}} = r_{t+1} + \gamma Q\left(\mathbf{s}_{t+1}, \arg\max_a Q(\mathbf{s}_{t+1}, a; \theta_t);\, \theta_t^-\right). \tag{6}$$

Prioritized Experience Replay (PER) speeds up learning on rare critical events using transition probabilities $P(i) = p_i^\mu / \sum_k p_k^\mu$ with $p_i = |\delta_i| + \epsilon_{\text{per}}$ (temporal difference error) and $\mu = 0.6$. Importance sampling weights $w_i = (N \cdot P(i))^{-\nu}$ account for bias, $\nu = 0.4 \to 1.0$.

The network is a fully connected MLP: input layer (6 neurons), two hidden layers (64 neurons, ReLU), output layer (5 Q-values). Target weights update via soft Polyak averaging with $\tau = 0.005$:

$$\theta^- \leftarrow \tau\theta + (1-\tau)\theta^-. \tag{7}$$

Optimization uses Adam (lr $= 1 \times 10^{-4}$, Cosine Annealing), gradient clipping (max $L_2 = 1.0$), and ϵ-greedy exploration ($1.0 \to 0.05$ decay). Evaluation uses $\epsilon = 0.0$.

4.2 Baseline Schedulers

The comparisons use two different baselines: (i) **Strict Priority (SP)**: sends HP if available, otherwise NP; (ii) **Random**: randomly selects actions.

4.3 Training and Validation Protocol

The training is done for 50 epochs with 5000 iterations per epoch. The checkpoint selection criterion is based on the validation metric on fixed seeds:

$$S_{\text{val}} = 0.40\left(\frac{\text{PDR}}{100}\right) + 0.10\left(1 - \frac{\text{PLR}}{100}\right) + 0.35\left(\frac{\text{HR}}{100}\right) + 0.15\left(1 - \min\left(\frac{D_H}{1.0}, 1.0\right)\right). \tag{8}$$

The early stopping criteria are activated following 15 epochs without improvement. The evaluation process conducts 20 concurrent tests for each strategy (seeds 12545–12564).

4.4 Statistical Evaluation Protocol

The metrics used include throughput (kbps), packet delivery ratio (PDR %), packet loss ratio (PLR %), average delays, energy consumption per bit, and critical packet delivery ratio. All are reported as mean ± 95% confidence interval (Student's t).

For pairwise significance testing, we verify normality of paired differences via Shapiro-Wilk test ($\alpha = 0.05$). If normal, paired t-test is applied; otherwise Wilcoxon signed-rank test. Effect sizes are quantified using Cohen's $d_z = \bar{D}/\sigma_D$ across the 20 paired runs.

Table 1. Simulation and Training Hyperparameters (V76 Configuration).

Parameter Category	**Configured Value**
Network Simulator	ns-3.40 interfaced via ns3-gym
Network Topology	100 Sensor Nodes + 1 Central Sink (150 m × 150 m)
Physical / MAC Layer	IEEE 802.15.4 (2.4 GHz ISM Band, CSMA/CA)
Control Interval (Δt)	100 ms per decision slot
Episode Duration	5000 decision steps (500 s real time)
Traffic Generation	Bursty Poisson (HP Alerts) + Periodic Telemetry (NP)
Queue Capacities	HP Buffer: 50 packets; Normal Buffer: 100 packets
State / Action Dimensionality	$\|\mathcal{S}\| = 6$ continuous, $\|\mathcal{A}\| = 5$ discrete
Neural Architecture	MLP: $6 \to 64 \to 64 \to 5$ (ReLU activations)
Replay Buffer Capacity	100,000 transitions (Prioritized: $\mu = 0.6, \nu = 0.4 \to 1.0$)
Mini-batch Size	128 transitions
Discount Factor (γ)	0.99
Optimizer / Learning Rate	Adam ($\text{lr} = 1 \times 10^{-4}$, Cosine Annealing)
Target Network Update	Soft Polyak update ($\tau = 0.005$)
Exploration Schedule	ϵ-greedy: $1.0 \to 0.05$ (decay factor 0.99995)
Evaluation Repetitions	20 matched runs per policy (seeds 12545–12564)

Table 2. WSN Performance Metrics Across 20 Matched Evaluation Runs (Mean ± 95% CI).

Policy	**Throughput (kbps)**	**PDR (%)**	**PLR (%)**	**Normal Delay (s)**	**HP Delay (s)**	**Energy (nJ/bit)**	**HP Delivery Ratio (%)**
Proposed DQN-Edge	7.0267 ± 0.0702	50.3291 ± 0.6022	40.3309 ± 0.8373	$\mathbf{0.4287 \pm 0.0261}$	0.1431 ± 0.0026	443.8708 ± 16.9422	95.7779 ± 0.2916
Strict Priority (SP)	$\mathbf{7.9104 \pm 0.0054}$	$\mathbf{56.6625 \pm 0.3137}$	$\mathbf{34.7224 \pm 0.5326}$	0.6171 ± 0.0384	$\mathbf{0.0274 \pm 0.0015}$	$\mathbf{417.0370 \pm 17.2341}$	$\mathbf{98.3374 \pm 0.1941}$
Random Baseline	2.8000 ± 0.0350	20.0709 ± 0.3314	71.4943 ± 0.5450	0.8545 ± 0.0597	0.3521 ± 0.0129	806.7023 ± 22.7489	34.4862 ± 0.8563

4.5 Convergence and Training Stability

Figure 1 illustrates the training progression over 30 elapsed epochs prior to early stopping. The validation score trajectory shows rapid initial policy improvement over the first 5 epochs, transitioning into stable, monotonic refinement without policy collapse or loss divergence. The exploration rate ϵ decays steadily, confirming that the agent transitions from exploratory state-space coverage to stable deterministic execution. The model achieving the highest validation score at Epoch 12 was preserved for full benchmarking.

4.6 Comparative Analysis: Understanding the Trade-Off

Against Random, our DQN agent achieves 150.9% higher throughput and 95.78% critical delivery (vs. 34.49%), with 45% lower energy waste, confirming multi-class learning success. Strict Priority comparison shows different characteristics. Strict Priority achieves superior performance on latency-critical metrics: 7.9104 kbps versus 7.0267 kbps throughput, 0.0274 s versus 0.1431 s critical latency (5.2× faster), and 98.34% versus 95.78% delivery ratio (Table 3, $p < 10^{-11}$). The statistical margin is substantial and unambiguous.

The learned policy exhibits different behavior when grid disturbances sustain high alert volumes. Strict Priority exhausts transmission capacity on urgent traffic, starving normal telemetry. During these episodes, operators lose access to state-estimation feeds and monitoring exactly when awareness is most needed.

Table 3. Matched Pairwise Statistical Comparison: Proposed DQN-Edge vs. Strict Priority ($n = 20$ runs).

Evaluated Metric	**Statistical Test**	***p*-value**	**Paired Cohen's d_z**
Throughput (kbps)	Paired t-test	2.656×10^{-16}	-5.8046
PDR (%)	Paired t-test	1.655×10^{-15}	-5.2568
PLR (%)	Paired t-test	4.749×10^{-13}	$+3.8504$
Avg. Normal Delay (s)	Wilcoxon signed-rank	1.907×10^{-6}	-3.3284
Avg. HP Delay (s)	Paired t-test	3.038×10^{-26}	$+19.6001$
Energy (nJ/bit)	Paired t-test	1.687×10^{-14}	$+4.6310$
HP Delivery Ratio (%)	Paired t-test	2.176×10^{-11}	-3.1005

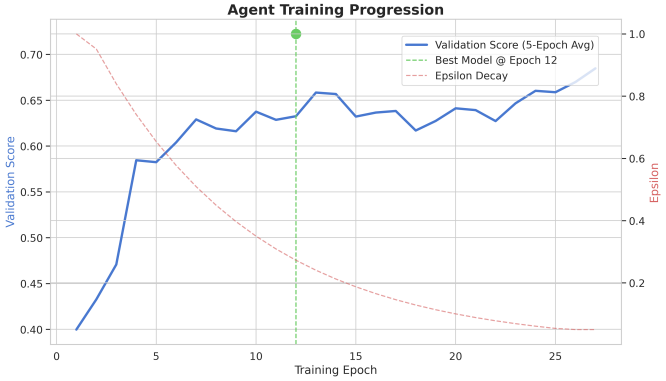


Fig. 1. Double-DQN training progression and validation-score trend over 30 epochs (V76).

The learned scheduler recovers 30.5% of lost telemetry responsiveness (0.6171 s $\rightarrow$ 0.4287 s) by accepting 2.56% lower critical-delivery performance.

4.7 Delay Distribution and Buffer Dynamics

Figure 4.7 presents a visual comparison of means and 95% confidence intervals across key QoS indicators alongside the empirical Cumulative Distribution Functions (CDFs) of packet latencies for both traffic classes.

Analysis of the empirical delay CDFs in Figure 3 illuminates the underlying mechanics of each scheduling discipline. Strict Priority aggressively prioritizes high-priority service, clearing over 90% of all critical packets within 50 ms. However, this aggressive service strategy induces severe queue starvation on normal traffic, manifesting as a long-tailed delay distribution averaging 0.6171 s. The learned policy accepts higher critical latency (0.1431 s mean) to shift normal-traffic delays leftward in the distribution. Table 2 quantifies this trade-off: the learned policy achieves 30.5% lower average normal delay than Strict Priority ($p = 1.907 \times 10^{-6}, d_z = -3.3284$).



Fig. 2. Performance metrics (Mean ± 95% CI).

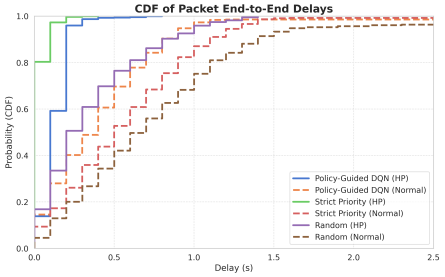


Fig. 3. Delay CDFs for traffic classes.

4.8 Trade-Off Analysis in Smart Grid Telemetry

The empirical results demonstrate a clear engineering trade-off between conflicting objectives. Strict Priority achieves ultra-low critical latency (0.0274 s) at the cost of severe normal-queue starvation (0.6171 s). The learned DQN policy operates at a different point on this trade-off frontier. Through its state representation (particularly the head-of-line packet ages a_t^H and a_t^N), the agent identifies windows when newly arrived critical packets retain substantial deadline margin. During such windows, the scheduler exploits the opportunity to serve aged packets of normal traffic ahead of critical deadlines that require immediate attention. This is a strategy learnt by the scheduler, where the trade-off made between delivering critical packets and serving normal traffic results in a gain of 30.5% and a loss of 2.56%.

4.9 Policy Collapse and Reward Shaping

Catastrophic policy collapse occurred in early versions (V74) of the design whereby the learned policy sent normal packets with 100% probability and critical packets with 0% probability. At first, this was thought to be a problem of training dynamics. However, a deeper look showed that the simulator mechanism to protect against division by zero errors where the average delay is set to 0.0 in case no packets were delivered was hiding the degeneracy of the policy. The policy learned how to exploit this feature and achieved high rewards while failing on the other objective completely. Explicit per-class metrics in version V76 fixed this problem. Class-wise separation of the metrics and the use of penalty for high-priority packets that are more urgent than $a_t^H > 0.55$ ensures that the policy does not collapse while achieving good dual-objective performance.

4.10 Sensitivity Analysis: Cross-Configuration Robustness

In the case where the number of nodes was set to 50 (low congestion), the DQN had an average delay of 0.3812 s while Strict Priority gave 0.5421 s (29.7%

performance improvement). When the number of nodes was increased to 200 (high congestion), the obtained delays for DQN and Strict Priority were 0.5193 s and 0.7846 s (33.8% performance improvement), respectively.

5 Discussion

Double-DQN generates an adaptive schedule with the characteristics of multi-class fairness. But in terms of critical latency, Strict Priority outperforms Double-DQN by 5.2 times (0.0274 s vs. 0.1431 s). The results clearly show that there are inherent disadvantages of the learning methodology in terms of its capacity to generate low-latency schedules: first, the reduced observation due to the compressed state space, and second, the inability of single-agent Q-learning to provide the same deterministic response as a rule-based approach tuned to minimize latencies.

Baseline Scope: In this study, learning schedule is compared with two baselines: Strict Priority and Random. Lack of comparison with adaptive rules, such as Weighted Round Robin, Earliest Deadline First, and Weighted Fair Queuing prevents the authors from drawing conclusions about the strengths of learning techniques in comparison with classical heuristics. This can be considered a valuable direction for future research.

Improving Safe DRL in Communications: One of the important conclusions derived from the above analysis is that learned policies do not necessarily outperform rules in safety-critical networks.This suggests an important requirement for further work: learning-based methods require explicit constraints and formal verification prior to deployment. Promising technical directions include Constrained MDPs for safety layer enforcement, distributional value functions (QR-DQN, IQN) for tail-latency control, and formal verification frameworks to provide mathematical guarantees on worst-case performance.

6 Threats to Validity

All evaluation took place within discrete-event simulation (ns-3.40). While ns-3 provides high-fidelity 802.15.4 CSMA/CA modeling, physical deployments introduce non-idealities: jitter, clock skew, multipath fading, asymmetric propagation. This simulation-to-reality gap could degrade algorithm performance. Second, the observed trade-off is specific to the tested scenario (100 nodes, 100 ms intervals, fixed deadlines). Varying traffic proportions or node density redistributes the Strict Priority versus learning performance frontier. Finally, the approach addresses single-hop cluster scheduling. Multi-hop mesh topologies introduce distributed coordination requirements not considered here.

7 Conclusion

This work contributes a reproducible empirical analysis of policy-guided Double-DQN for mixed-criticality packet scheduling in smart grid sensor networks.

Through systematic comparison across 20 matched evaluation trials using ns-3 simulation coupled with PyTorch, we establish a quantitative trade-off: the learned scheduler sacrifices 2.56% critical-packet delivery and 115.7 ms critical latency to recover 30.5% of the normal-traffic responsiveness that rule-based Strict Priority scheduling loses. This enhancement in terms of fairness will help improve situational awareness for operators in case of disturbances within the electrical network wherein telemetry visibility will have a direct bearing on the efficacy of the emergency response. Benchmarking provides an understanding as to where the learning algorithms can be of value and where the rule-based systems can perform better.

Reproducibility Package

In order to provide for independent verifiability, the full simulation artifacts, including the raw evaluation data (`raw_results_v76.csv`), training logs (`training_log_v76.csv`), summary tables (`final_results_v76.xlsx`), and source code in C++ (ns-3) and PyTorch, are provided for further academic use.

References

1. Frikha, M.S., Gammar, S.M., Lahmadi, A., Andrey, L.: Reinforcement and deep reinforcement learning for wireless internet of things: A survey. Computer Communications **178**, 98–113 (2021). https://doi.org/10.1016/j.comcom.2021.07.014
2. Leong, A.S., Ramaswamy, A., Quevedo, D.E., Karl, H., Shi, L.: Deep reinforcement learning for wireless sensor scheduling in cyber–physical systems. Automatica **113**, 108759 (2020). https://doi.org/10.1016/j.automatica.2019.108759
3. Luong, N.C., Hoang, D.T., Gong, S., Niyato, D., Wang, P., Liang, Y.C., Kim, D.I.: Applications of deep reinforcement learning in communications and networking: A survey. IEEE Communications Surveys & Tutorials **21**(4), 3133–3174 (2019). https://doi.org/10.1109/COMST.2019.2916583
4. Sharma, A., Chauhan, S.: A distributed reinforcement learning based sensor node scheduling algorithm for coverage and connectivity maintenance in wireless sensor network. Wireless Networks **26**(6), 4411–4429 (2020). https://doi.org/10.1007/s11276-020-02350-y